

Autocomplétion

Recherche de recettes

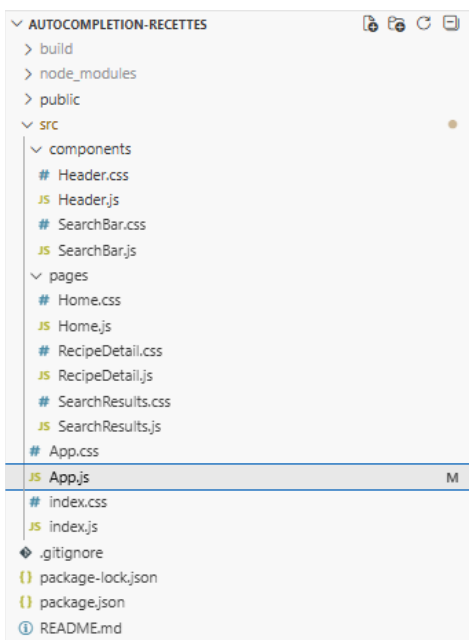
Des propositions qui s'adaptent à tes demandes

Descriptif du projet

L'outil de barre de recherche est un outil très utilisé sur les sites. Elle englobe un élément très important : l'autocomplétion.

L'**autocomplétion** va permettre de faire une recherche plus ou moins précise selon son paramétrage. Il s'agit du mécanisme qui permet de proposer à l'utilisateur des résultats pour sa recherche, en fonction de ce qu'il tape dans l'input. Les résultats s'adaptent à chaque fois que l'utilisateur entre un caractère supplémentaire.

Dans ce projet, tu vas créer une application de recherche de recettes de cuisine en utilisant l'API gratuite [TheMealDB](#). Cette API contient des milliers de recettes avec leurs ingrédients, instructions, photos et plus encore.



Ton application doit contenir les composants et pages suivantes :

- Une page d'accueil avec une barre de recherche centrale (style moteur de recherche)
- Une page de résultats de recherche qui affiche toutes les recettes correspondant à la requête
- Une page de détails d'une recette avec toutes les informations (ingrédients, instructions, photo...)
- Une barre de recherche dans le header présente sur toutes les pages

Figure 1: exemple de structure de fichier



La barre de recherche doit proposer une autocomplétion intelligente :

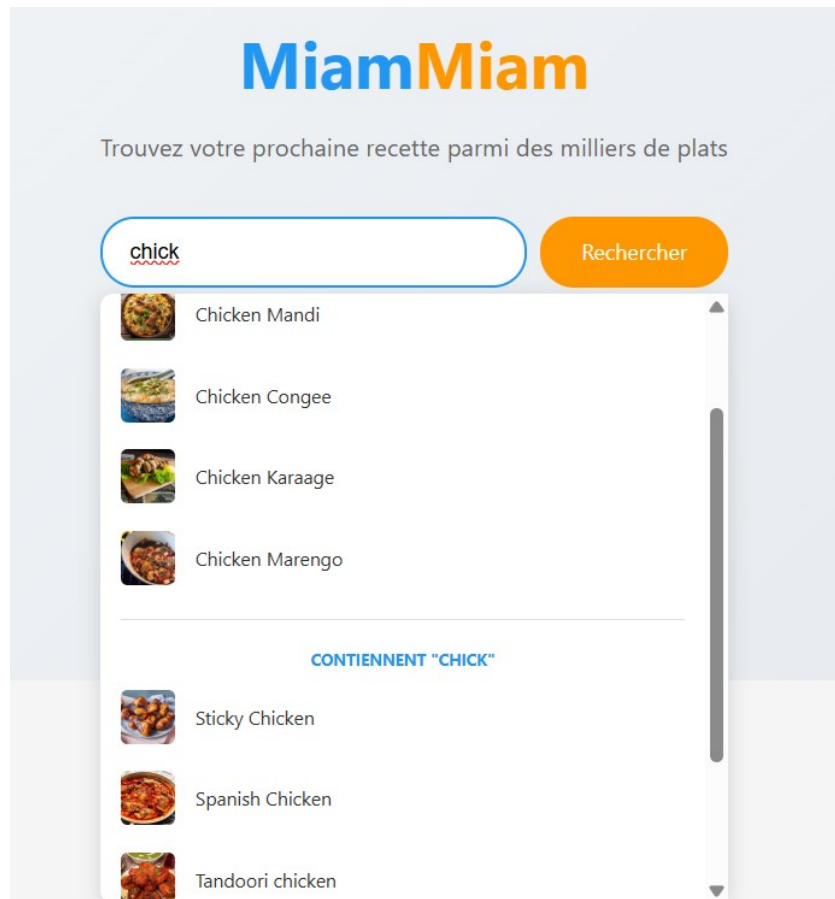


Figure 2: Voici un exemple d'autocomplétion.

Au fur et à mesure que l'utilisateur tape, des suggestions doivent apparaître, divisées en **deux parties distinctes** :

- En premier : les résultats qui commencent exactement par ce que l'utilisateur tape
- En second : les résultats qui contiennent la recherche de l'utilisateur (mais ne commencent pas par)

💡 **Exemple :** Si l'utilisateur tape "chic", tu dois afficher en premier les recettes qui commencent par "chic" (Chicken, Chick pea...), puis celles qui contiennent "chic" ailleurs (BBQ Chicken, Spicy chicken...).



Job 00 - Configuration et découverte de l'API

Créer sur GitHub un répertoire nommé **"runtrackReact"**. Dans ce répertoire, créer un dossier **"autocomplete-recettes"**.

Installer ReactJS :

```
npx create-react-app autocomplete-recettes
```

Découvrir l'API TheMealDB :

L'API TheMealDB est gratuite et ne nécessite pas de clé API. Voici les endpoints principaux que tu vas utiliser :

- Rechercher par nom : <https://www.themealdb.com/api/json/v1/1/search.php?s={query}>
- Détails d'une recette : <https://www.themealdb.com/api/json/v1/1/lookup.php?i={id}>
- Rechercher par première lettre : <https://www.themealdb.com/api/json/v1/1/search.php?f={letter}>

Tester l'API :

Ouvre ton navigateur et teste cette URL :

```
https://www.themealdb.com/api/json/v1/1/search.php?s=chicken
```

Observe la structure JSON retournée :

- meals : tableau contenant les recettes
- idMeal : identifiant unique
- strMeal : nom de la recette
- strCategory : catégorie (Beef, Chicken, Dessert...)
- strArea : origine (British, Italian, Mexican...)



- strInstructions : instructions de préparation
- strMealThumb : URL de l'image
- strIngredient1 à strIngredient20 : ingrédients
- strMeasure1 à strMeasure20 : quantités

Job 01 - Page d'accueil et barre de recherche basique

Créer la page d'accueil avec une barre de recherche centrale.



Figure 3: exemple de la page d'accueil

À faire :

1. Créer un composant Home.js pour la page d'accueil
2. Centrer verticalement et horizontalement un formulaire de recherche
3. Créer un composant SearchBar.js avec un input et un bouton
4. Utiliser useState pour gérer la valeur de l'input
5. Au clic sur le bouton ou à la validation du formulaire, rediriger vers `/search?q={query}`
6. Styliser la page pour ressembler à Google (simple, épuré, centrée)



Conseil : Utilisez React Router pour la navigation entre les pages. Installez-le avec : `npm install react-router-dom`



Job 02 - Page de résultats de recherche

Créer la page qui affiche les résultats de recherche.

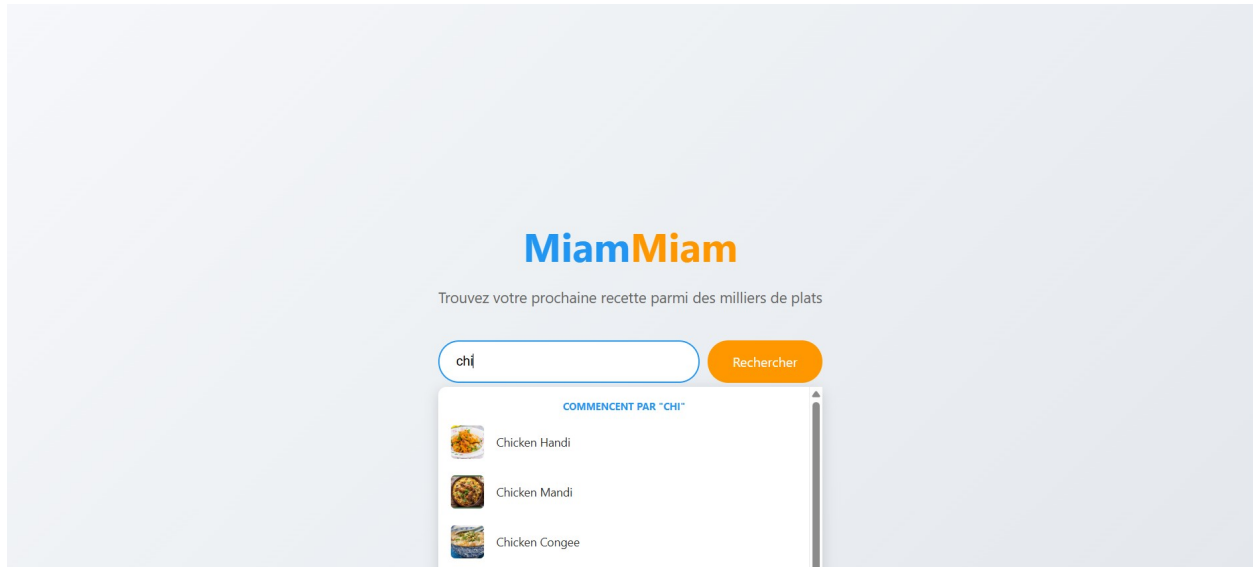


Figure 4: Exemple de page avec une recherche en cours.

À faire :

1. Créer un composant SearchResults.js
2. Récupérer le paramètre 'q' de l'URL avec useParams() ou useSearchParams()
3. Utiliser useEffect pour appeler l'API TheMealDB avec la requête
4. Utiliser useState pour stocker les résultats
5. Afficher les recettes dans une grille (cards avec image, nom, catégorie)
6. Chaque card doit être cliquable et rediriger vers /recipe/:id
7. Gérer le cas où aucun résultat n'est trouvé
8. Afficher un loader pendant le chargement



À faire :

1. Créer un composant RecipeDetail.js
2. Récupérer l'ID de la recette depuis l'URL avec useParams()
3. Appeler l'API pour récupérer les détails complets de la recette
4. Afficher : image, nom, catégorie, origine, instructions
5. Afficher la liste des ingrédients avec leurs quantités
6. Ajouter un bouton "Retour aux résultats"
7. Gérer le cas où la recette n'existe pas (404)

Astuce pour les ingrédients :

L'API retourne les ingrédients et mesures séparément (strIngredient1, strMeasure1, etc.). Tu dois les combiner :

```
const getIngredients = (meal) => {  
  const ingredients = [];  
  for (let i = 1; i <= 20; i++) {  
    const ingredient = meal[`strIngredient${i}`];  
    const measure = meal[`strMeasure${i}`];  
  
    if (ingredient && ingredient.trim()) {  
      ingredients.push(`${measure} ${ingredient}`);  
    }  
  }  
  return ingredients;  
};
```



Job 04 - Autocomplétion basique

Ajouter une autocomplétion simple à la barre de recherche.

À faire :

1. Dans SearchBar.js, ajouter un state pour les suggestions
2. Utiliser useEffect pour appeler l'API à chaque changement de l'input
3. Afficher une liste de suggestions sous l'input
4. Limiter à 10 suggestions maximum
5. Au clic sur une suggestion, remplir l'input et lancer la recherche
6. Cacher les suggestions quand l'input est vide
7. Ajouter un debounce pour ne pas appeler l'API à chaque frappe (attendre 300ms)

Exemple de debounce :

```
useEffect(() => {  
  const timer = setTimeout(() => {  
    if (searchTerm.length >= 2) {  
      fetchSuggestions(searchTerm);  
    }  
  }, 300); // Attendre 300ms après la dernière frappe  
  
  return () => clearTimeout(timer);  
}, [searchTerm]);
```

💡 **Astuce :** Pour éviter trop d'appels API, commence la recherche seulement si l'utilisateur a tapé au moins 2 caractères.



Job 05 - Autocomplétion intelligente (2 parties)

Améliorer l'autocomplétion pour séparer les résultats en deux catégories.

Logique à implémenter :

Diviser les suggestions en deux groupes :

- Partie 1 : Recettes qui commencent par la recherche (startsWith)
- Partie 2 : Recettes qui contiennent la recherche mais ne commencent pas par (includes)

Exemple de code :

```
const filterSuggestions = (meals, query) => {
  if (!meals) return { startsWith: [], contains: [] };

  const lowerQuery = query.toLowerCase();

  const startsWith = meals.filter(meal =>
    meal.strMeal.toLowerCase().startsWith(lowerQuery)
  );

  const contains = meals.filter(meal => {
    const lowerName = meal.strMeal.toLowerCase();
    return lowerName.includes(lowerQuery) && !lowerName.startsWith(lowerQuery);
  });

  return {
    startsWith: startsWith.slice(0, 5),
    contains: contains.slice(0, 5)
  };
};
```

Affichage :

- Afficher d'abord les résultats "startsWith" avec un label ou style différent
- Ajouter un séparateur visuel (ligne horizontale ou label "Autres résultats")
- Afficher ensuite les résultats "contains"
- Si une catégorie est vide, ne pas afficher le séparateur



Job 06 - Navigation au clavier et UX avancée

Améliorer l'expérience utilisateur avec la navigation au clavier.

Fonctionnalités à ajouter :

- Navigation avec les flèches ↑ et ↓ dans les suggestions
- Sélection avec la touche Entrée
- Fermeture des suggestions avec Échap
- Highlight de la suggestion active
- Fermeture des suggestions au clic en dehors

Exemple pour détecter les touches :

```
const handleKeyDown = (e) => {
  if (e.key === 'ArrowDown') {
    e.preventDefault();
    setSelectedIndex(prev =>
      Math.min(prev + 1, suggestions.length - 1)
    );
  } else if (e.key === 'ArrowUp') {
    e.preventDefault();
    setSelectedIndex(prev => Math.max(prev - 1, 0));
  } else if (e.key === 'Enter' && selectedIndex >= 0) {
    e.preventDefault();
    handleSelectSuggestion(suggestions[selectedIndex]);
  } else if (e.key === 'Escape') {
    setSuggestions([]);
  }
};
```

💡 **Bonus :** Ajoute une icône de chargement pendant que l'API répond, et une icône de loupe dans l'input.



Job 07 - Header global et fonctionnalités bonus

Finaliser l'application avec un header présent sur toutes les pages.

À faire :

1. Créer un composant Header.js avec la barre de recherche
2. Intégrer ce header dans App.js pour qu'il soit présent partout
3. Ajouter le logo/nom de l'application dans le header
4. Rendre le logo cliquable pour retourner à l'accueil
5. S'assurer que l'autocomplétion fonctionne depuis n'importe quelle page

Fonctionnalités bonus (optionnelles) :

- Filtrer par catégorie (Beef, Chicken, Dessert, Seafood...)
- Filtrer par origine (British, Italian, Mexican...)
- Historique des recherches récentes (localStorage)
- Recettes favorites avec localStorage
- Mode sombre / clair
- Animations sur les suggestions (fade in/out)
- Partage de recette (bouton partager)
- Impression de la recette (bouton imprimer)

⚠ **Important** : Teste ton autocomplétion avec différents cas : recherches courtes, recherches longues, caractères spéciaux, recherches sans résultats.

Compétences visées

- Utiliser des événements JavaScript (onChange, onKeyDown, onClick)
- Manipuler le DOM avec React
- Maîtriser fetch() et les appels API asynchrones
- Gérer les états avec useState (input, suggestions, loading)
- Utiliser useEffect pour les effets de bord (appels API)



- Implémenter un debounce pour optimiser les performances
- Filtrer et trier des données en JavaScript
- Créer une navigation fluide avec React Router
- Gérer les événements clavier pour une meilleure UX

Rendu

Le projet est à rendre sur <https://github.com/prenom-nom/runtrackReact>, dossier "autocompletion-recettes".

Base de connaissances

- Documentation React - useState - <https://react.dev/reference/react/useState>
- Documentation React - useEffect - <https://react.dev/reference/react/useEffect>
- React Router - <https://reactrouter.com/>
- TheMealDB API - <https://www.themealdb.com/api.php>
- MDN - Fetch API - https://developer.mozilla.org/fr/docs/Web/API/Fetch_API
- MDN - Événements clavier - <https://developer.mozilla.org/fr/docs/Web/API/KeyboardEvent>
- JavaScript - String.startsWith() - https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String/startsWith
- JavaScript - String.includes() - https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String/includes
- JavaScript - Debounce - <https://www.freecodecamp.org/news/javascript-debounce-example/>